

Lecture 2: GitHub Actions CI for a Python Project

A Step-by-Step Setup from Scratch

1 Introduction and Goal of the Lecture

In Lecture 1, we studied why Continuous Integration (CI) and Continuous Delivery (CD) are needed, especially for complex systems. In this lecture, we move from theory to practice. Our goal is to start from an empty folder on a local machine and finish with a Python project that automatically runs tests on GitHub whenever we push code or open a pull request.

We will do this by gradually building a GitHub Actions workflow. As we write each part of the workflow, we will also define the terminology: workflow, event, job, runner, step, and action. By the end, you should be able to both run and explain a basic CI pipeline.

2 Local Python Project Setup

Before we introduce GitHub Actions, we need a small Python project to automate. We will create a minimal structure with one module and one test.

2.1 Create a Project Folder

On your machine, open a terminal and run:

```
mkdir github-actions-demo
cd github-actions-demo
```

This directory will become our Git repository and our Python project.

2.2 Create a Virtual Environment (Optional but Recommended, I do not need it myself)

To keep dependencies isolated:

```
python -m venv .venv
source .venv/bin/activate    # On Windows: .venv\Scripts\activate
```

2.3 Create a Simple Python Module

Create a file `app.py` with a trivial function we can test:

```
def add(a: int, b: int) -> int:
    return a + b

if __name__ == "__main__":
    print(add(2, 3))
```

2.4 Create a Test Using `pytest`

Create a `tests` folder and a test file:

```
mkdir tests
```

Now create `tests/test_app.py`:

```
from app import add

def test_add():
    assert add(2, 3) == 5
```

Now, you have to add the project root to `PYTHONPATH` so that you can import `app` like that as module. Create a file named `conftest.py`. It has to be named such way, otherwise `pytest` will not be able to find it. In the file, add the project root to the python path accordingly. **Make sure the file is in the project root folder**

```
import sys
from pathlib import Path

_root = Path(__file__).resolve().parent
if str(_root) not in sys.path:
    sys.path.insert(0, str(_root))
```

2.5 Add a Requirements File

Create `requirements.txt`:

```
pytest==8.0.0
```

Install the dependencies locally:

```
pip install -r requirements.txt
```

Run tests locally to confirm everything works:

```
pytest
```

At this point, we have a minimal Python project and a passing test. CI will simply automate running `pytest` in the cloud.

3 Initialize a Git Repository and Push to GitHub

GitHub Actions workflows live in a GitHub repository, so we must first initialize Git and push our code. Also make sure to have a `.gitignore` file. Search on the internet and find a sample `.gitignore` list with python and pytest projects etc. This is what I have:

```
# Byte-compiled / optimized / DLL files
__pycache__/
*.py[cod]
*$py.class

# Virtual environments
.venv/
venv/
```

```

env/
.env/

# Pytest
.pytest_cache/
.coverage
htmlcov/

# Distribution / packaging
dist/
build/
*.egg-info/
*.egg

# IDE
.idea/
.vscode/
*.swp
*.swo

# OS
.DS_Store
Thumbs.db

# Local / env files (secrets)
*.local
.env.local

```

3.1 Initialize Git and Make the First Commit

In the project directory:

```

git init
git add .
git commit -m "Initial commit: simple Python project with tests"

```

3.2 Create a GitHub Repository and Add Remote

On GitHub, create an empty repository (e.g., `github-actions-demo`). Follow the instructions GitHub shows to add the remote. For example:

```

git remote add origin https://github.com/<your-username>/github-actions-demo.git
git branch -M main
git push -u origin main

```

Now the code is in GitHub, and we are ready to add CI using GitHub Actions.

4 Understanding GitHub Actions at a High Level

Before writing YAML, we briefly define the main components. A **workflow** is an automated process defined in a YAML file inside `.github/workflows/`. A workflow is triggered by an

event, such as a `push` or `pull_request`. Each workflow contains one or more **jobs**. A job runs on a **runner**, which is a virtual machine provided by GitHub (e.g., `ubuntu-latest`) or self-hosted. Each job is composed of **steps**. Steps are executed sequentially and can either run shell commands or use reusable **actions**, which are pre-packaged tasks such as checking out code or setting up Python.

We will internalize these definitions by writing a workflow from scratch.

5 Step 3: Creating the First Workflow File

GitHub Actions looks for workflows in the `.github/workflows` directory. We will create that folder and add our first workflow file.

5.1 Create the Workflow Directory and File

From the project root:

```
mkdir -p .github/workflows
```

Create a file named `python-ci.yml` inside `.github/workflows/`. Remember, `.git` and `.github` are two different things. The former is about git repo, config, git history whereas the latter is about githubs own things such as actions and other github related features.

5.2 Define the Workflow Name and Trigger

We start with the top of the YAML file:

```
name: Python CI

on:
  push:
  pull_request:
```

The `name` field is just a human-readable label that will appear in the GitHub Actions UI. The `on` section defines **events**. Here we say: run this workflow whenever code is pushed or a pull request is opened. This immediately connects CI to the integration events we discussed in Lecture 1.

6 Step 4: Adding a Job and Understanding Runners

Next, we define one job called `test`. A job represents a logical unit of work, such as running tests.

```
jobs:
  test:
    runs-on: ubuntu-latest
```

The key points are:

- `jobs`: top-level collection of jobs in this workflow.
- `test`: the name of this specific job.

- `runs-on`: specifies the **runner** environment. Here, `ubuntu-latest` means the job will run on a GitHub-hosted Linux virtual machine.

Conceptually, when a push occurs, GitHub allocates a temporary Ubuntu virtual machine and executes the job there. This VM is isolated from your local machine and destroyed after the job finishes.

7 Step 5: Adding Steps and Using Actions

Now we must tell the job what to do, using **steps**. Each step either uses an **action** or runs a shell command.

7.1 Step 1: Check Out the Code

The runner must fetch the repository’s code. GitHub provides a standard action for this:

```
steps:
  - name: Check out repository
    uses: actions/checkout@v4
```

Here, `uses` means “use an existing action”. `actions/checkout@v4` is a published, versioned action maintained by GitHub. This action:

- Clones the repository.
- Checks out the correct commit associated with the event.

Without this step, the runner would have an empty filesystem. Find any related gitactions from this link <https://github.com/actions>

7.2 Step 2: Set Up Python

We need a Python interpreter on the runner. GitHub provides another action:

```
- name: Set up Python
  uses: actions/setup-python@v5
  with:
    python-version: '3.10'
```

Here, `with` passes configuration inputs to the action. We are telling the action to install and configure Python 3.10 on the runner. Once this step completes, subsequent steps in the job can run commands like `python` and `pip`.

7.3 Step 3: Install Dependencies

Next, we install the dependencies listed in `requirements.txt`:

```
- name: Install dependencies
  run: |
    pip install --upgrade pip
    pip install -r requirements.txt
```

The `run` keyword executes shell commands directly on the runner. Here we upgrade `pip` and install the project’s dependencies. This mirrors what we did locally before running tests.

7.4 Step 4: Run Tests

Finally, we run the test suite:

```
- name: Run tests
  run: pytest
```

If `pytest` exits with a non-zero exit code (because tests failed), the job is marked as failed, and the whole workflow fails. GitHub will show a red “X” in the repository UI. This is where CI enforces stability: no failing test should silently enter the main branch.

7.5 Full Workflow File So Far

Putting everything together, our `python-ci.yml` file now looks like:

```
name: Python CI

on:
  push:
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Check out repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.10'

      - name: Install dependencies
        run: |
          pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run tests
        run: pytest
```

At this point, we have a complete, minimal CI workflow: whenever we push or open a pull request, GitHub will spin up a Linux VM, check out our repository, install Python, install dependencies, and run tests.

8 Step 6: Committing and Triggering the Workflow

To activate the workflow, we commit and push the new YAML file.

8.1 Commit the Workflow File

From the project root:

```
git add .github/workflows/python-ci.yml
git commit -m "Add GitHub Actions workflow for Python CI"
git push
```

8.2 Observe the Workflow Run on GitHub

Navigate to the repository on GitHub and click on the “Actions” tab. You should see a new workflow run triggered by the recent push. Click on it to see the job and step-level logs.

This is where students can connect the YAML definition to the actual execution trace. Each step in the YAML corresponds to a section in the logs. If something fails (for example, a typo in the YAML or missing dependency), GitHub will highlight the failing step.

9 Step 7: Understanding CI Feedback and Failure Modes

When tests pass, the workflow is marked green. When tests fail, the workflow is marked red. This information can be integrated into pull requests as a status check. Many teams configure GitHub so that pull requests cannot be merged unless CI passes.

If we intentionally break the test, for example by changing

```
assert add(2, 3) == 5

to

assert add(2, 3) == 6
```

and then commit and push, the next workflow run will fail at the “Run tests” step. This demonstrates CI’s role in preventing regressions from entering the shared main branch.

10 Small Enhancements

Once the basic pipeline is understood, we can briefly mention two common enhancements without going deep.

10.1 Running on a Specific Branch

To run CI only on the main branch, we can refine the trigger:

```
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]
```

This ensures that only changes targeting main will trigger the workflow.

10.2 Caching Dependencies (Performance Optimization)

Installing dependencies on every run can be slow. GitHub provides a cache action to speed up repeated runs. The idea is to reuse previously downloaded packages when the `requirements.txt` file has not changed. A simple example:

```
- name: Cache pip
  uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: ${ runner.os }-pip-${ hashFiles('**/requirements.txt') }
    restore-keys: ${ runner.os }-pip-
```

This step would be added before installing dependencies.

11 Summary and Conceptual Connections

In this lecture, we implemented a complete GitHub Actions CI pipeline for a simple Python project. We started with a local project, initialized Git, pushed to GitHub, and then created a workflow file step-by-step. As we wrote the YAML, we explained the core concepts:

We saw that a workflow is a YAML-defined automation process stored under `.github/workflows/`. It is triggered by events such as `push` and `pull_request`. Within the workflow, we defined a job that runs on a GitHub-hosted runner (`ubuntu-latest`). Inside the job, we defined steps that used actions (`actions/checkout`, `actions/setup-python`) and shell commands (`pip install`, `pytest`). Each step's success or failure contributed to the overall status of the workflow.

12 In Class Work

Finish everything above and send me screenshots on discord to get in-class-activity points.